Posture Coach: Build Guide

Oct 10, 2026 · @gilad

Overview

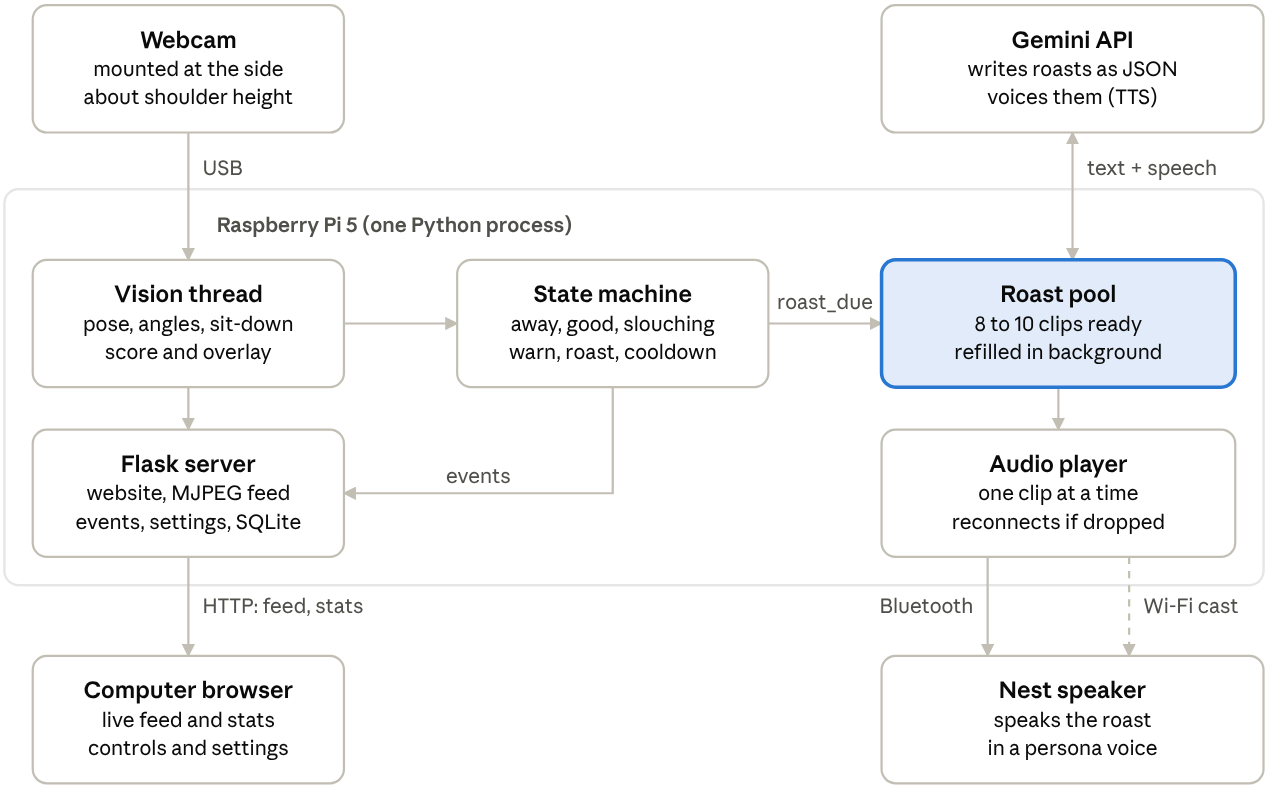
Posture Coach watches the user from the side with a webcam on a Raspberry Pi 5, and when they slouch too long, the Google Nest roasts them with an insult written by Gemini. Everything runs on the Pi; the computer only opens the website in a browser.

The end-to-end flow:

1. A side-mounted USB webcam feeds frames to the Pi.
2. OpenCV and MediaPipe find body landmarks (ear, shoulder, hip) and detect when someone sits down, which starts a session.
3. The Pi measures neck and torso angles and compares them to the user's calibrated upright posture.
4. A state machine decides when slouching has lasted long enough to deserve a roast, with a cooldown so it doesn't repeat constantly.
5. The Pi picks a Gemini-written roast, converts it to speech, and plays it on the Nest over Bluetooth.
6. A Flask server on the Pi hosts the website: a live annotated camera feed and session stats, viewed from the computer's browser.

Gemini is kept out of the real-time path: roasts are generated and converted to audio ahead of time, so a slouch gets a response in about a second instead of waiting on an API call.

Everything runs on the Pi; Gemini fills a roast pool ahead of time



system architecture · primary path and backup

The dashed line is the backup audio route: if Bluetooth fails during judging, the Pi casts the clip to the Nest over Wi-Fi instead.

Architecture and design decisions

The Pi runs one Python process: a Flask app with a few background threads. One process is simpler than several, and only one program can open the webcam at a time anyway.

What runs on the Pi

Component	Runs as	Job
Camera + vision	Background thread	Grabs frames, runs pose detection, computes angles, draws the overlay
State machine	Called by the vision thread	Decides sit-down, good, slouch, away, and when a roast is due

Component	Runs as	Job
Roast worker	Background thread	Keeps a pool of ready-to-play roast audio clips, refilled from Gemini
Audio player	Background thread with a queue	Plays clips on the Nest one at a time, never overlapping
Flask web server	Main thread	Serves the website, the live video stream, stats, and settings
SQLite	File on the Pi	Stores sessions, posture samples, and roasts played

How audio reaches the Nest

The Pi 5 has Bluetooth Classic, so it can pair with the Nest exactly like a phone does and play audio to it. Put the Nest in pairing mode from the Google Home app (the speaker's audio settings have a paired-devices option), then pair from the Pi with `bluetoothctl`. After that, the Nest appears as an audio output on the Pi, and any WAV file you play comes out of it.

Route	How it works	Strengths	Weaknesses
Bluetooth (primary)	Pi plays audio to the Nest as a Bluetooth speaker	Matches your design; no network needed between Pi and Nest	Bluetooth audio on Linux can be finicky; the link can drop when idle
Wi-Fi cast (backup)	Pi uses `pychromecast` to tell the Nest to play a clip served by Flask	No pairing; usually very reliable on a shared network	Needs the Pi and Nest on the same network without device isolation

Build the cast backup early. If Bluetooth acts up during judging, switching should be one setting on the website.

Where Gemini fits

Gemini does two jobs, both done ahead of time rather than at the moment of the slouch:

1. **Write the roasts.** A text model returns a batch of short roasts as structured JSON. The prompt includes session context (how many times they've slouched, how long the session has run, how bad the angle got), so roasts can escalate: "Slouch number five. Your spine has filed a complaint."
2. **Voice the roasts.** A Gemini text-to-speech model turns each roast into audio, which the Pi saves as a WAV file in the pool.

The roast worker keeps around 8 to 10 clips ready. When one is played, it generates a replacement in the background. If Gemini is slow or offline, the system falls back to pre-recorded roasts and a local text-to-speech engine, so the demo still works.

Google's API has moved quickly: the Python SDK is `google-genai` (the older `google-generativeai` package is deprecated), and Google made its newer Interactions API the default in June 2026. Follow the current Gemini API quickstart rather than older tutorials, and pick model IDs from Google's models page on the day.

Keep the roasts playful

The insults are the fun of the project, but strangers (judges) will be on the receiving end. Put these rules in the prompt:

- Roast the **posture only**, never appearance, body, identity, or intelligence.
- Keep it PG and under about 15 words, so it's quick and punchy when spoken.
- Escalate in tiers: mild teasing at first, more dramatic (still PG) for repeat offenders.

Add a "nice mode" toggle on the website that switches to encouragement instead of roasts. Gemini's safety filters may also block harsher wording, so a tame prompt is more reliable as well.

Escalation logic

Slouching for about 10 seconds turns the website's status indicator yellow. Past about 30 seconds, the Nest plays a roast. After a roast, a cooldown (around 60 to 90 seconds) prevents back-to-back insults. Tune these numbers during testing.

Parts and software checklist

The whole build is a Pi, a webcam, and the Nest; most of the risk is in software setup, so do the installs before the event.

Hardware

- [ ] Raspberry Pi 5 with its official 27 W USB-C power supply (weaker supplies cause throttling and USB issues)
- [ ] Active cooler for the Pi 5; pose detection runs the CPU hard all day and it will throttle without one
- [ ] microSD card (32 GB+) with Raspberry Pi OS (64-bit), plus a spare card with the same image
- [ ] USB webcam and a way to mount it at the side, roughly at shoulder height and 1 to 2 m away (a small tripod or clamp)
- [ ] Google Nest Mini or Nest Audio, set up in a Google Home account on a teammate's phone
- [ ] Mobile hotspot or travel router so the Pi, computer, and Nest share one network
- [ ] Ethernet cable or a keyboard and HDMI cable for recovering the Pi if Wi-Fi setup fails

Software on the Pi

- [ ] Python 3 virtual environment with `opencv-python`, `mediapipe`, `numpy`, `flask`
- [ ] `google-genai` (the current Gemini SDK) and a Gemini API key from Google AI Studio, stored in a `.env` file that is never committed
- [ ] Bluetooth audio working with the OS's default audio system (Pi OS uses PipeWire); test with a WAV file before the event
- [ ] `pychromecast` for the Wi-Fi backup path
- [ ] A local text-to-speech engine for offline fallback (for example `espeak-ng` or Piper)
- [ ] SQLite (built into Python)

Team setup

- [ ] A shared Git repo with folders: `vision/`, `server/`, `voice/`, `web/`
- [ ] SSH access to the Pi for every teammate (enable SSH and set a known hostname like `posture.local`)
- [ ] Everyone can develop on their own laptop with a webcam or recorded video, then deploy to the Pi

Do before the event

- [ ] Install everything on the Pi and confirm `import mediapipe` works (if pip has trouble on the Pi, find out now, not at hour 1)
- [ ] Pair the Pi with the Nest over Bluetooth and play a test WAV
- [ ] Make one Gemini text call and one text-to-speech call from the Pi
- [ ] Record 2 to 3 short videos of teammates sitting and slouching from the side, for developing vision without the live setup

Interface contracts (agree on these in hour 0)

Agree on these boundaries before anyone codes, so all four people can build in parallel against fakes. Changing one later requires telling everyone.

Vision → state machine

The vision module turns a frame into one reading. Everything downstream only sees readings, never frames:

```
{
  "ts": 1760101234.5,
  "person": true,
  "seated": true,
  "neck_angle": 18.4,
  "torso_angle": 7.1,
  "score": 0.86
}
```

Angles are in degrees from vertical. `score` is 0 to 1, where 1 matches the calibrated upright posture.

State machine → voice

The state machine emits events; the voice module only reacts to `roast_due`:

```
{ "event": "roast_due", "slouch_count": 5, "session_minutes": 23,
"worst_neck_angle": 41 }
```

Other events: `session_start`, `warn`, `good`, `session_end`. The context fields let the next Gemini batch write roasts that reference the session.

Flask routes (website ↔ Pi)

```
GET  /                 the website
GET  /video_feed       live annotated camera feed (MJPEG stream)
GET  /events           server-sent events: each reading summary, state
change, and roast played
GET  /api/stats        current session: good %, slouch count, roasts, longest
good streak
GET  /api/history      readings for the chart
POST /api/calibrate    start the 5-second upright calibration
GET  /api/settings     thresholds, nice mode, audio route (bluetooth or cast)
POST /api/settings     update them
POST /api/test-roast   play a roast right now (for demos and testing)
```

Fakes to build first

- **Recorded video as the camera.** The vision code reads a video file instead of the webcam, so the vision person can work without the Pi.
- **Fake readings.** A script that produces a scripted sequence of good and slouching readings, so the server and website people can build before vision works.
- **Fake player.** A player that prints the roast text instead of playing audio, so the server can be tested without Bluetooth.

Build step 1: Side-view posture detection

A side view is a real advantage: you can measure actual neck and torso angles instead of guessing from a front-facing picture. The vision code compares those angles to the user's own calibrated upright posture.

Step 1.1: Camera loop with landmarks

Open the webcam with OpenCV at 640×480 and run each frame through MediaPipe's pose landmarker (start with the lite model on the Pi). You need four landmarks on the side facing the camera: **ear, shoulder, hip, knee**.

MediaPipe reports both left and right landmarks even when one side is hidden. Pick the side whose landmarks have the higher visibility score, since the far side is partly a guess.

Draw the ear–shoulder and shoulder–hip lines on the frame with the current angles printed beside them. This overlay is your debugging tool and the centerpiece of the website's live feed.

Checkpoint: on the Pi, the overlay tracks a seated person smoothly. Print the frames per second; aim for 10 or more. If it's lower, shrink the frame or run detection on every second frame.

Step 1.2: Angles that capture slouching

MediaPipe gives coordinates normalized from 0 to 1 in each direction. **Convert them to pixels first** (multiply x by the frame width and y by the height). Otherwise the angles are distorted, because the frame is wider than it is tall. Remember that image y increases downward.

Neck angle (forward head posture): the angle of the ear–shoulder line from vertical. It grows as the head juts forward.

```
\theta_{\text{neck}} = \arctan\!\left( \frac{\left| x_{\text{ear}} - x_{\text{shoulder}} \right|}{y_{\text{shoulder}} - y_{\text{ear}}} \right)
```

Torso angle (hunching or leaning): the angle of the shoulder–hip line from vertical.

```
\theta_{\text{torso}} = \arctan\!\left( \frac{\left| x_{\text{shoulder}} - x_{\text{hip}} \right|}{y_{\text{hip}} - y_{\text{shoulder}}} \right)
```

In code, use `atan2` rather than a plain arctangent of a fraction, so a near-zero denominator can't blow up.

Before building on these, print both angles while a teammate sits straight, then slouches, then leans back. Confirm which ones move and by roughly how much with your camera placement.

Step 1.3: Detect when someone sits down

From the side, a seated person's thigh is close to horizontal. Measure the hip–knee line's angle from horizontal:

```
\theta_{\text{thigh}} = \arctan\!\left( \frac{\left| y_{\text{knee}} - y_{\text{hip}} \right|}{\left| x_{\text{knee}} - x_{\text{hip}} \right|} \right)
```

Treat the person as seated when this angle stays small (for example, under about 30°) for about 2 seconds. Starting a session only after they've settled avoids roasting someone who is still sitting down.

A desk can hide the knee. If the knee's visibility is low, fall back to: a person is present, the hip stays within a small region of the frame, and the torso is upright enough, all for a few seconds.

Checkpoint: a teammate walks in and sits; the overlay shows "seated" after about 2 seconds and "away" when they stand up and leave.

Step 1.4: Calibration

When a session starts (or the user clicks Calibrate), ask them to sit up straight for 5 seconds and average both angles over that window. Those averages are the baseline values θ_0 for the neck and torso. Store them so they survive a restart.

Step 1.5: Turn angles into a score

Measure how far each angle has drifted past its baseline, relative to a tolerance $\Delta\theta$ (try about 20° and tune it):

```
s_{\text{neck}} = 1 - \min\!\left(1,\; \frac{\max\left(0,\;
\theta_{\text{neck}} - \theta_{0,\text{neck}}\right)}{\Delta\theta}\right)
```

Compute the torso score the same way, and use the lower of the two as the overall score, so either kind of slouch counts. Then smooth it so a single noisy frame can't trigger a roast:

```
\bar{s}_t = \alpha\, s_t + (1 - \alpha)\, \bar{s}_{t-1}, \qquad \alpha
\approx 0.2
```

Step 1.6: Classify with hysteresis

Use two thresholds so the state doesn't flicker when the score hovers near one value: switch to slouching when the smoothed score drops below about 0.7, and back to good only when it rises above about 0.8. When the person isn't detected or isn't seated, the state is away, and nothing is ever roasted while away.

Checkpoint: run your recorded videos through the code; the labels match what everyone sees, with no flickering.

Build step 2: Flask server on the Pi

The server ties vision, the state machine, and voice together, and serves the website. Its main challenge is sharing one camera and one stream of data safely between threads.

Step 2.1: App structure

- The **camera thread** loops forever: grab a frame, run vision, draw the overlay, encode the frame as JPEG once, and store the latest JPEG and reading in a shared object protected by a lock.
- Flask routes only **read** from that shared object. They never touch the camera.
- Run Flask on `0.0.0.0` port 5000 with threading enabled, so the computer can reach it at `http://posture.local:5000`.
- **Turn off Flask's auto-reloader** on the Pi. It starts the app twice, and the second copy can't open the camera, which produces confusing errors.

Checkpoint: the Pi serves a test page, and the computer's browser can reach it by hostname.

Step 2.2: Live video feed

Serve `/video_feed` as an MJPEG stream: a response that keeps sending the latest JPEG as a new part, about 10 times a second. A plain `<img>` tag on the website displays it with no extra code.

Because the camera thread encodes each frame once, extra viewers cost almost nothing. Don't encode inside the route, or every open browser tab multiplies the CPU load.

Checkpoint: the computer's browser shows the live annotated feed with under a second of delay.

Step 2.3: State machine

The state machine owns every decision about when to roast. Vision only reports readings; voice only does what it's told.

State	Enters when	Action on entry	Leaves when
Away	No person, or not seated	End the session if one was running	Seated for about 2 s

State	Enters when	Action on entry	Leaves when
Calibrating	Session starts, or Calibrate is clicked	Record 5 s of baseline angles	Baseline saved
Good	Score above the upper threshold	Emit `good`	Score drops below the lower threshold
Slouching	Score below the lower threshold	Start a timer	Good again, or timer hits 10 s
Warned	Slouching for 10 s	Emit `warn` (website turns yellow)	Good again, or slouching hits 30 s
Roasted	Slouching for 30 s, and no cooldown active	Emit `roast_due`; start the cooldown	Good again

Two rules to get right:

- **Debounce the return to good.** Require about 2 s of good posture before resetting the slouch timer, or a brief sit-up dodges the roast forever.
- **Cooldown blocks roasts, not tracking.** During the cooldown the state still updates and the website still shows slouching; only the audio waits.

Checkpoint: feed the fake readings through it; `roast_due` fires at exactly the right times, and never twice inside the cooldown.

Step 2.4: Live updates and logging

- Push events to the website with **server-sent events** on `/events`: a simple one-way stream that Flask supports with a generator, and the browser reads with `EventSource`. Send a reading summary about twice a second, plus every state change and roast.
- Log readings (sampled, not every frame), state changes, and roasts played to SQLite. Give each thread its own database connection, since SQLite connections shouldn't be shared across threads.
- Compute session stats from the log: percentage of time with good posture, slouch count, roasts played, and longest good streak.

Step 2.5: Start on boot

Run the app as a `systemd` service so it starts automatically when the Pi powers on and restarts if it crashes. At the venue, plugging in the Pi should be all it takes to get the demo running.

Checkpoint: reboot the Pi; the website comes back by itself within about a minute.

Build step 3: Gemini roasts and the Nest

The voice module keeps a pool of finished roast clips ready, so playing one is instant. Gemini fills the pool in the background, and local fallbacks keep it full if Gemini fails.

Step 3.1: Bluetooth audio to the Nest (do this in hour 1)

This is the riskiest piece of hardware setup, so prove it first.

1. In the Google Home app, open the Nest's settings and turn on Bluetooth pairing mode.
2. On the Pi, use `bluetoothctl` to scan, then **pair**, **trust**, and **connect** to the Nest. Trusting it lets the Pi reconnect automatically later.
3. Make the Nest the default audio output and play a test WAV from the command line.

Two quirks to plan for:

- The Bluetooth link can drop after a period of silence. Before playing each clip, check that the Nest is connected and reconnect if not.
- A Bluetooth speaker often clips the first fraction of a second while the link wakes up. Add about half a second of silence to the start of every clip.

Checkpoint: a test WAV plays on the Nest, and plays again after 10 minutes of silence.

Step 3.2: Generate roasts with Gemini

Ask a Gemini text model for a batch of roasts as **structured JSON** (the API can enforce a schema), for example a list of objects with the roast text and its intensity tier. Structured output means no fragile parsing of free text.

What the prompt should contain:

- The rules from the architecture section: posture only, PG, under about 15 words.
- The session context from the `roast_due` event: slouch count, session length, worst angle.

- The tier wanted: mild for the first few slouches, more dramatic for repeat offenders.
- The last few roasts already used, so it doesn't repeat itself.

Validate what comes back before using it: drop anything too long, empty, or containing words from a short blocklist.

Checkpoint: one call returns 5 valid, varied roasts that reference the session context.

Step 3.3: Voice the roasts with Gemini text-to-speech

Send each roast to a Gemini text-to-speech model and save the audio as a WAV file. Gemini's TTS models take a style instruction along with the text, so you can give the voice a persona (a disappointed drill sergeant, a dramatic sports commentator). That's a cheap way to make the demo memorable.

The TTS models return raw audio samples, so you need to write a proper WAV header with the sample rate and format the docs specify. Check Google's models page for the current TTS model; they are marked preview and names change.

Checkpoint: a generated roast plays on the Nest in the chosen voice.

Step 3.4: The roast pool

- A background worker keeps 8 to 10 ready clips per tier. When a clip is played, it generates a replacement.
- When a `roast_due` event arrives, the player takes a clip from the right tier immediately; there is no waiting on the network.
- Refresh part of the pool with the latest session context every few roasts, so roasts stay relevant ("That's slouch number seven").
- Play clips through a queue, one at a time. If a clip is already playing, skip the new one rather than stacking them.

Step 3.5: Fallbacks

- **Gemini text fails:** use a hand-written list of about 20 roasts.
- **Gemini TTS fails:** voice the roast with a local engine on the Pi (lower quality, but it works offline).
- **Bluetooth fails:** switch the audio route setting to Wi-Fi cast. The Pi tells the Nest (through `pychromecast`) to play the clip from a Flask URL.

Pre-generate and save a full pool of Gemini clips before judging, so even a total loss of internet still sounds good.

Checkpoint: with the Pi's internet disconnected, a slouch still gets a roast.

Build step 4: The website

The website is what judges see first, so it should look finished even though it's a single page. Flask serves it from the Pi; the computer just opens it in a browser.

Step 4.1: Layout

- **Live feed** (left, large): the `/video_feed` stream with the angle overlay.
- **Status** (right, top): a big colored indicator for the current state (green good, yellow warned, red roasted, gray away), plus the live neck and torso angles.
- **Last roast**: the text of the most recent roast, shown as a caption when it plays. Judges can read it, and it makes for great screenshots.
- **Session stats**: good-posture percentage, slouch count, roasts delivered, longest good streak.
- **Score chart**: posture score over the session, with roast moments marked.

Step 4.2: Controls

- **Calibrate** button.
- **Test roast** button that plays one immediately; essential for the demo.
- **Settings**: the warn and roast times, cooldown, nice mode on or off, and the audio route (Bluetooth or cast).
- **System status**: camera frame rate, Nest connection, and whether Gemini is reachable, so problems are visible at a glance.

Step 4.3: Live updates

Listen to `/events` with the browser's `EventSource` and update the indicator, angles, chart, and captions as events arrive. Load `/api/stats` and `/api/history` once on page load so a refresh shows the whole session.

Checkpoint: with the fake readings running, the indicator, chart, and roast captions all update live with no page refresh.

Step 4.4: Stretch goals (only after everything else works)

- A "hall of shame": the session's best roasts, with the slouch angle that earned each one.
- A Gemini-written end-of-session report card.
- A leaderboard comparing teammates' good-posture percentages.

Team roles for 4 people

Each person owns one layer end to end and builds against the fakes until the real pieces exist. Whoever is most comfortable with Linux should take the voice role, since Bluetooth audio setup is the biggest unknown.

Role	Owns	Builds	Hands off to
1. Vision	`vision/`	Pose landmarks, side selection, angles, sit-down detection, calibration, score, hysteresis (Build step 1)	State machine, via readings
2. Server	`server/`	Flask app, camera thread, MJPEG feed, state machine, SQLite, server-sent events, systemd service, fake readings (Build step 2)	Voice via `roast_due`, website via routes
3. Voice	`voice/`	Bluetooth to the Nest, Gemini roast prompt and schema, Gemini TTS, roast pool, player queue, all fallbacks including cast (Build step 3)	Plays what the server asks for
4. Website + demo lead	`web/`	Website, controls, settings, live chart, captions, pitch, demo script, backup video (Build step 4)	Owns the final integration test and the pitch

Pairings when someone is blocked or done early

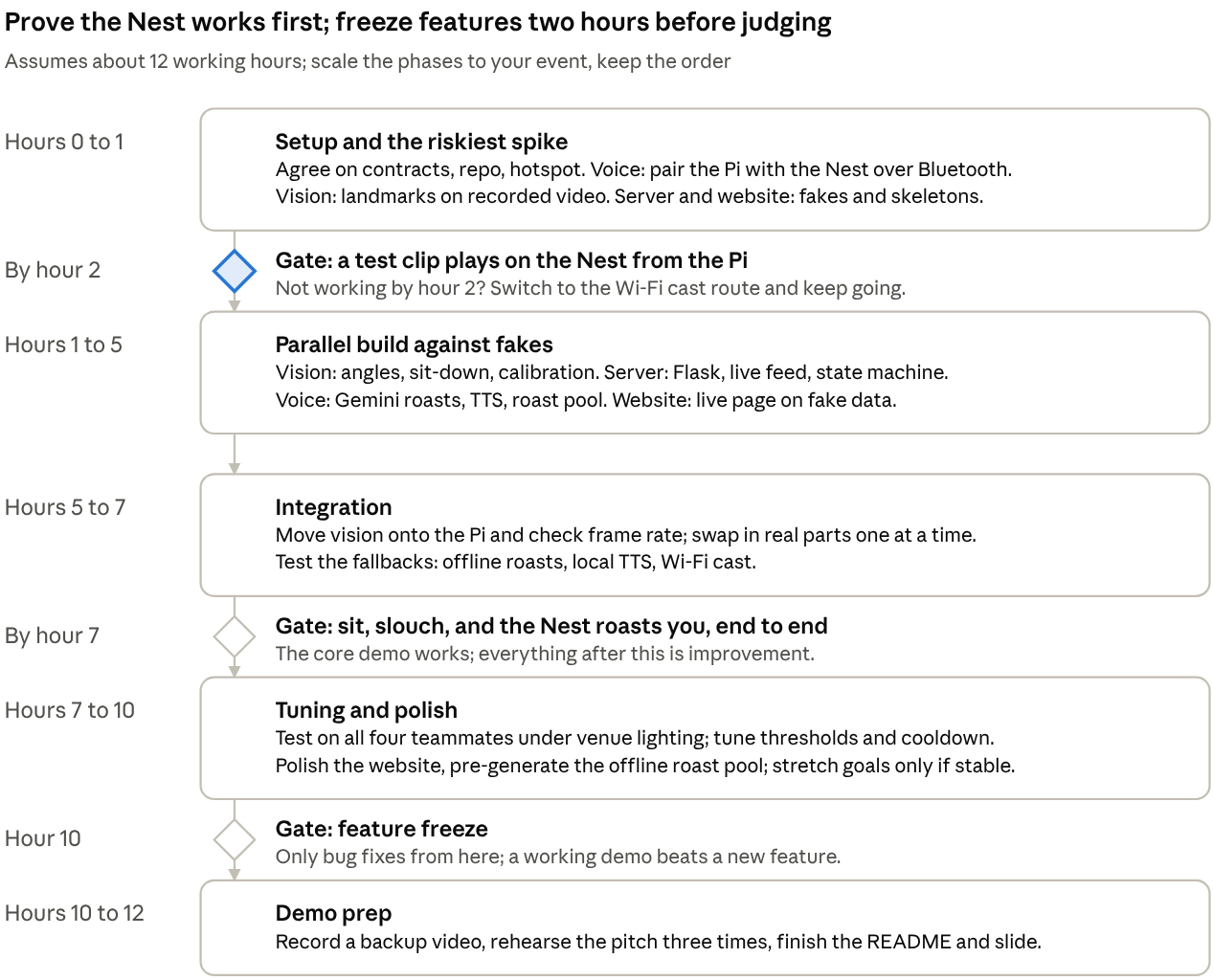
- Vision and server pair on moving the vision code onto the Pi and measuring the frame rate there.

- Voice and website pair on the roast captions and the test-roast button.
- Whoever finishes first writes the hand-written fallback roasts and pre-generates the offline clip pool.

Rules that keep four people from colliding

- One folder per person; commit small and often.
- Only one person deploys to the Pi at a time; announce it before restarting the service.
- Contracts only change with everyone's agreement, announced out loud.
- Every two hours, a 5-minute standup: what works, what's blocked, what's next.

Timeline for the day



day plan · 5 phases, 3 gates

If a gate isn't met on time, take its fallback instead of pushing the schedule: every later phase depends on the one before it.

Risks, fallbacks, testing, and pitch

The three things most likely to sink the demo are Bluetooth audio, the Pi's frame rate, and venue internet, so each has a tested fallback before lunch.

Risks and fallbacks

Risk	Sign it's happening	Fallback
Bluetooth to the Nest drops or won't pair	Roast shows on the website but the Nest is silent	Reconnect before each clip; switch the audio route to Wi-Fi cast
Pose detection too slow on the Pi	Under about 8 frames per second, laggy overlay	Lite model, smaller frames, detect on every second frame; posture changes slowly, so 5 fps of detection is still enough
Venue Wi-Fi blocks device-to-device traffic	Computer can't reach the Pi; cast can't find the Nest	Run everything on your own hotspot or travel router from the start
No internet, or Gemini is slow or rate-limited	Pool stops refilling	Pre-generated clip pool, hand-written roasts, local TTS
Gemini declines a prompt or returns something off-tone	Empty or rejected batch	Keep the prompt PG; validate and blocklist; fall back to the hand-written list
Side-view landmarks unreliable (desk hides hips or knees, baggy clothes)	Flickering seated state, wild angles	Raise the camera or angle it slightly down; use the visibility-based fallback for sit-down
Pi overheats or browns out	Throttling warnings, random reboots	Active cooler and the official power supply

Testing checklist before judging

- [ ] Each teammate sits, calibrates, and slouches; it works for all four body types and chair heights
- [ ] Test in the venue's actual lighting and with the camera where it will sit during the demo
- [ ] Stand up mid-session: the state goes to away and nothing is roasted
- [ ] Disconnect the Pi's internet: roasts still play from the pool
- [ ] Turn the Nest's Bluetooth off mid-session: the website shows it, and switching to cast works
- [ ] Reboot the Pi: the service comes back by itself
- [ ] Run a 10-minute session end to end without touching the code
- [ ] Record a backup demo video once everything works

Pitch for software judges (about 3 minutes)

1. **Problem (20 s):** students spend hours hunched over laptops, and polite posture reminders are easy to ignore. Insults are not.
2. **Live demo (90 s):** a judge sits down; the system notices, calibrates, and they slouch on purpose. The Nest roasts them, and the roast appears on the website next to the live angle overlay.
3. **Architecture (45 s):** one slide of the system diagram. Emphasize the side-view geometry and per-user calibration, the debounced state machine, and keeping the LLM out of the real-time path with a pre-generated, context-aware roast pool.
4. **What's next (25 s):** posture history and trends, multiple user profiles, and roast personalities.

Expect these questions and have short answers ready: why a side view (it measures real neck and torso angles), how you avoid false alarms (calibration, smoothing, hysteresis, the away state), and what happens if Gemini is down (pre-generated pool plus offline fallbacks).